САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Практическая/Лабораторная работа 1

Выполнил:

Саид Наваф

Группа

К3341 ПОТОК 1.1

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Используйте любой boilerplate, который считаете наиболее подходящим для ваших нужд (можно взять и мой)
2. С учётом имеющихся данных по результатам ДЗ1 и ДЗ2 реализуйте:
    - модели
    - представления
    - контроллеры
3. В результате выполнения лабораторной работы вы должны продемонстрировать полностью работающее API согласно вашего варианта

Необходимо сделать отчёт по шаблону

Пример бойлерплейта: express-typeorm-boilerplate Пример реализации REST API: express-microservices-example

Ход работы

Описание хода работы:

1. Настройка проекта

Инициализирован Go-модуль с зависимостями (gin-gonic, pgx, jwt)

Настроена multi-stage Docker-сборка для оптимизации размера образа

Создана структура проекта по принципу Clean Architecture:

- `cmd/api` — точка входа
- `internal/handler` — HTTP обработчики
- `internal/repository` — работа с базой данных
- `internal/model` — структуры данных
- `internal/middleware` — JWT middleware

2. Проектирование базы данных

Созданы миграции PostgreSQL со следующими таблицами:

users — пользователи (id, email, phone, role, password_hash)

employers — работодатели (user_id, company_name)

applicants — соискатели (user_id, first_name, last_name)

categories — категории вакансий (id, name, slug)

vacancies — вакансии (employer_id, category_id, title, description, salary, city)

resumes — резюме (applicant_id, desired_position, experience, about)

applications — отклики (vacancy_id, resume_id, status)

3. Реализация эндпоинтов

Аутентификация:

- `POST /api/v1/auth/register` — регистрация пользователя
- `POST /api/v1/auth/login` — получение JWT-токена

Публичные эндпоинты:

- `GET /api/v1/categories` — список категорий
- `GET /api/v1/vacancies` — список всех активных вакансий
- `GET /api/v1/vacancies/{id}` — детали вакансии
- `GET /api/v1/resumes` — список резюме
- `GET /api/v1/resumes/{id}` — детали резюме

Защищённые эндпоинты (требуют JWT):

- `GET /api/v1/profile` — получить профиль текущего пользователя
- `POST /api/v1/vacancies` — создать вакансию (только employer)
- `POST /api/v1/resumes` — создать резюме (только applicant)
- `POST /api/v1/applications` — откликнуться на вакансию (только applicant)
- `GET /api/v1/applications` — мои отклики (только applicant)
- `GET /api/v1/employer/applications` — отклики на вакансии работодателя
- `PATCH /api/v1/applications/{id}/status` — изменить статус отклика (только employer)

4. Реализация безопасности

JWT middleware проверяет токен в заголовке `Authorization: Bearer <token>`

Role-based access control: проверка роли пользователя для каждого эндпоинта

Password hashing с использованием bcrypt

5. Наполнение базы демо-данными

Добавлены тестовые данные:

4 категории (IT, Marketing, Sales, Design)

3 пользователя (employer, applicant, admin)

3 вакансии с полной информацией

2 резюме

3 отклика на вакансии

6. Тестирование

Все эндпоинты протестированы через:

Браузер (для GET-запросов)

PowerShell Invoke-WebRequest (для POST/GET/PATCH запросов)

Проверка корректности JWT-авторизации

Проверка валидации ролей

скриншоты

1. Список вакансий

localhost:8080/api/v1/vacancies


```
автоформатировать
{
  "id": 11,
  "employer_id": 1,
  "category_id": 2,
  "title": "Marketing Manager",
  "description": "Продвижение IT-продуктов",
  "requirements": "Опыт digital-маркетинга, знание аналитики",
  "working_conditions": "Гибридный формат работы",
  "salary_min": 100000,
  "salary_max": 180000,
  "experience_required": 2,
  "city": "Санкт-Петербург",
  "address": "Кронверкский пр., 49",
  "is_active": true,
  "created_at": "2026-04-15T11:48:54.733187Z"
},
{
  "id": 10,
  "employer_id": 1,
  "category_id": 1,
  "title": "Junior Frontend Developer",
  "description": "Верстка сайтов на React",
  "requirements": "Знание HTML/CSS/JavaScript, базовое понимание React",
  "working_conditions": "Офис, обучение от менторов",
  "salary_min": 80000,
  "salary_max": 120000,
  "city": "Москва",
  "address": "ул. Примерная, 10",
  "is_active": true,
  "created_at": "2026-04-15T11:48:54.733187Z"
},
{
  "id": 9,
  "employer_id": 1,
  "category_id": 1,
  "title": "Senior Go Developer",
  "description": "Разработка высоконагруженных сервисов на Go",
  "requirements": "Опыт работы с Go от 3 лет, знание PostgreSQL, Docker",
  "working_conditions": "Удалённая работа, гибкий график",
  "salary_min": 200000,
  "salary_max": 350000,
  "experience_required": 3,
  "city": "Санкт-Петербург",
  "address": "Кронверкский пр., 49",
  "is_active": true,
  "created_at": "2026-04-15T11:48:54.733187Z"
},
{
```


2. Список резюме

localhost:8080/api/v1/resumes


```
автоформатировать
[
  {
    "id": 4,
    "applicant_id": 1,
    "desired_position": "Go Backend Developer",
    "expected_salary": 150000,
    "experience": 2,
    "about": "Студент ИТМО, знаю Go и PostgreSQL.",
    "is_active": true,
    "created_at": "2026-04-15T11:29:52.675248Z"
  },
  {
    "id": 3,
    "applicant_id": 1,
    "desired_position": "Go Backend Developer",
    "expected_salary": 150000,
    "experience": 2,
    "about": "Студент ИТМО, знаю Go и PostgreSQL.",
    "is_active": true,
    "created_at": "2026-04-15T11:27:05.856144Z"
  },
  {
    "id": 2,
    "applicant_id": 2,
    "desired_position": "Go Developer",
    "expected_salary": 200000,
    "experience": 3,
    "about": "????? ?????? ?????",
    "is_active": true,
    "created_at": "2026-04-15T06:01:41.365594Z"
  },
  {
    "id": 1,
    "applicant_id": 1,
    "desired_position": "Middle Go Developer",
    "expected_salary": 150000,
    "experience": 2,
    "about": "???? ?????? ??? ? ????? ?????",
    "is_active": true,
    "created_at": "2026-04-15T05:53:19.861444Z"
  }
]
```


3. Категории

localhost:8080/api/v1/categories

втоформатировать


```
{
  "id": 1,
  "name": "Информационные технологии",
  "slug": "it"
},
{
  "id": 2,
  "name": "Маркетинг и реклама",
  "slug": "marketing"
},
{
  "id": 8,
  "name": "Медицина и фармацевтика",
  "slug": "medicine"
},
{
  "id": 9,
  "name": "Образование и наука",
  "slug": "education"
},
{
  "id": 3,
  "name": "Продажи",
  "slug": "sales"
},
{
  "id": 5,
  "name": "Производство",
  "slug": "production"
},
{
  "id": 6,
  "name": "Строительство",
  "slug": "construction"
},
{
  "id": 7,
  "name": "Транспорт и логистика",
  "slug": "logistics"
},
{
  "id": 4,
  "name": "Финансы и бухгалтерия",
  "slug": "finance"
},
{
  "id": 10,
  "name": "Юриспруденция",
```


5. Успешный запуск Docker

```
PS C:\Users\Admin\ITMO-ACS-Backend-2026-A\БР1.1\БР1.1_Саид_Наваф_лаб1> docker-compose ps
NAME             IMAGE                  COMMAND                  SERVICE    CREATED          STATUS                     PORTS
job-search-api   job-search-api:latest  "./api"                  api        18 minutes ago   Up 18 minutes              0.0.0
0:8080->8080/tcp, [::]:8080->8080/tcp
job-search-db    postgres:15-alpine     "docker-entrypoint.s…"   postgres   8 hours ago      Up About an hour (healthy) 0.0.0
0:5432->5432/tcp, [::]:5432->5432/tcp
PS C:\Users\Admin\ITMO-ACS-Backend-2026-A\БР1.1\БР1.1_Саид_Наваф_лаб1>
```

6. Защищённый эндпоинт

localhost:8080/api/v1/profile


```
{"error":"Authorization header is required"}
```

Вывод

Вывод по работе:

В ходе выполнения лабораторной работы был разработан полнофункциональный REST API для системы поиска работы.

Достигнутые результаты:

1. Реализована архитектура приложения с разделением на слои (handler → repository → database)
2. Настроена работа с PostgreSQL через драйвер pgx
3. Внедрена JWT-аутентификация с middleware
4. Реализована система ролей (employer/applicant) с разграничением доступа
5. Созданы все необходимые CRUD-операции
6. Приложение контейнеризировано в Docker с multi-stage сборкой
7. База данных наполнена демо-данными для демонстрации